Discovering RegEx in Python

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

What is a Regular Expression?

A **Regular Expression (RegEx)** is a **set of symbols** (a string) used to **identify a specific group of strings**, enabling the detection of patterns within a text.

Example use cases:

- finding emails;
- extracting numbers;
- identifying specific words.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Anchors

- ^ — **Start-of-line anchor**

Matches the position at the **beginning** of a string or line.

Example: ^Hello matches any line that **starts with** “Hello”.

- $ — **End-of-line anchor**

Matches the position at the **end** of a string or line.

Example: world$ matches any line that **ends with** “world”.

- ^Hello$

Matches the **exact line** “Hello” and nothing else.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Quantifiers/1

- * — **Zero or more**

Matches **0 or more occurrences** of the preceding element.

Example: abc* matches "ab", "abc", "abcc", "abccc", etc.

- + — **One or more**

Matches **1 or more occurrences** of the preceding element.

Example: abc+ matches "abc", "abcc", "abccc", but **not** "ab".

- ? — **Zero or one**

Matches **0 or 1 occurrence** of the preceding element.

Example: abc? matches "ab" or "abc".

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Quantifiers/2

- {n} — **Exactly n times**

Examples: abc{3} matches "abccc"

- {n,} — **At least n times**

Example: abc{2,} matches "abcc", "abccc", "abcccc", etc.

- {n,m} — **Between n and m times**

Example: abc{2,4} matches "abcc", "abccc", or "abcccc".

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Wildcards

- . — **Dot (Any character)**

Matches **any single character** except newline (\n) by default.

Example: a.c matches "abc", "axc", "a1c", "a c", "a-c", "a;c", but not "ac" or "abbc".

- .* — **Zero or more of any character**

Matches **any number of any characters**, including none.

Example: a.*c matches "ac", "abc", "axyzc", etc.

- .+ — **One or more of any character**

Matches **at least one** of any character.

Example: a.+c matches "abc", "axyzc", but **not** "ac".

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operators/1

- | — **Alternation (OR)**

Matches **either** the expression before or after the operator.

Example: cat|dog matches "cat" or "dog".

- (...) — **Grouping / Capture**

Groups expressions together and captures matched text.

Captured groups can **be reused or referenced later**.

Example: (abc)+ matches "abc", "abcabc", etc.

- (?:...) — **Non-capturing group**

Groups expressions **without capturing** the matched text.

Useful for applying quantifiers or alternation **without storing the result**.

Example: (?:ab|cd)+ matches "ab", "cd", "abcd", etc.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Operators/2

- (?=...) — **Positive lookahead**

Asserts that what follows the current position **matches** the expression.

Example: a(?=b) matches "a" only if it is **followed by** "b".

- (?!...) — **Negative lookahead**

Asserts that what follows **does not match** the expression.

Example: a(?!b) matches "a" only if it is **not followed by** "b".

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its
ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Character class/1

- [...] — **Character class**

Matches **any single character** of the characters inside the brackets.

Example: gr[ae]y matches "gray" or "grey".

Ranges:

- [a-z]

Matches **any lowercase letter** from a to z.

- [0-9]

Matches **any digit** from 0 to 9.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Character class/2

Negation:

- [^abc]

Matches any character **except** a, b, or c.

The ^ **inside** brackets means "not".

Combining Classes:

- [a-zA-Z0-9_]

Matches any **letter, digit, or underscore**.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Character class/3

- **\d — Digit**

Matches any **single digit** (equivalent to [0-9])

Example: \d\d matches "42", "99"

- **\D — Non-digit**

Matches any **character that is NOT a digit** (equivalent to [^0-9])

Example: \D+ matches "abc", "--"

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Character class/4

- **\w — Word character**

Matches any **letter, digit, or underscore** (equivalent to [a-zA-Z0-9_])

Example: \w+ matches "hello123", "my_variable"

- **\W — Non-word character**

Matches any **character that is NOT a word character** (equivalent to [^a-zA-Z0-9_])

Example: \W+ matches spaces, punctuation, symbols like "!", "#", " ".

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Character class/5

- \s — **Whitespace character**

Matches **any space, tab, newline, etc.**

Example: \s+ matches " " or "\n\t"

- \S — **Non-whitespace character**

Matches **any character that is NOT a space, tab, or newline**

Example: \S+ matches words and symbols, skipping whitespace.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Boundary

- **\b — Word boundary**

Matches the **position between a word character and a non-word character** (or at the start/end of a string next to a word character). Used to match whole words.

Example: \bcat\b matches "cat" within "the cat sat" but not "caterpillar“.

- **\B — Non-word boundary**

Matches a position **that is NOT a word boundary**. Useful for finding substrings inside words.

Example: \Bcat\B matches "educate", but not "cat" or "catfish"

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Using RegEx in Python

Python includes the **re** module for regular expressions.

Example:

```
import re
text:str = "My email is marco@gmail.com"
result:list[str] = re.findall(r'\S+@\S+', text)
print(result) # Output ['marco@gmail.com']
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Practical RegEx Examples/1

Python includes the re module for regular expressions.

Example:

1. Starts with capital?

```
import re
text:str = "Rome Paris"
result = re.match(r'[A-Z][a-z]+', text)
print(result.group()) # Output "Rome"
```

This regex means:

- [A-Z]: Match **one uppercase letter**
- [a-z]+: Match **one or more lowercase letters**

So it matches a capitalized word.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Practical RegEx Examples/2

2. Find numbers:

```
import re
text:str = "I have 20 cats and 3 dogs"
numbers:list[str] = re.findall(r'\d+', text)
print(numbers) # Output ['20', '3']
```

This regex means:

- \d: matches a digit (0-9)
- +: matches one or more consecutive digits

So it matches all the integers found in the text.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Python re module – Common Methods/1

Method	Description	Example Usage	Returns
`re.match()`	Checks for a match **only at the beginning** of the string	`re.match(r"\d+", "123abc")`	Match object or None
`re.search()`	Scans the string for **the first match** anywhere	`re.search(r"\d+", "abc123")`	Match object or None
`re.fullmatch()`	Checks if **the entire string** matches the regex pattern	`re.fullmatch(r"\d+", "123")`	Match object or None
`re.findall()`	Returns a list of **all non-overlapping matches**	`re.findall(r"\d+", "a1b2c3")`	`['1', '2', '3']`
`re.finditer()`	Returns an **iterator of match objects** for all matches	`re.finditer(r"\d+", "a1b2c3")`	Iterator of Match objects
`re.sub()`	**Replaces** matches with a string	`re.sub(r"\d", "#", "a1b2")`	`"a#b#"`
`re.split()`	**Splits** a string at each match	`re.split(r"\d+", "a1b2c3")`	`['a', 'b', 'c', '']`
`re.compile()`	Compiles a regex into a **RegexObject** for reuse	`pattern = re.compile(r"\d+")`	Compiled pattern object

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Python re module – Common Methods/2

Method	Example	Method	Example
re.match()	`re.match(r"\d+", "123abc")` `# Matches: 123`	re.search()	`re.search(r"\d+", "abc123")` `# Matches: 123`

re.findall()	`re.findall(r"\d\d", "12345")`
	`# Output: ['12', '34']`
		re.finditer()	`for m in re.finditer(r"\d+", "a1b2"):`
			`  print(m.group())`
			`  # Matches: 1 (first iter)`
			`  # Matches: 2 (second iter)`

re.sub()	`re.sub(r"\d", "#", "a1b2")` `# Output: 'a#b#'`	re.split()	`re.split(r"\d+", "a1b2c3")` `# Output: ['a', 'b', 'c', '']`

re.compile()	`pattern = re.compile(r"\d+")`
	`pattern.findall("a1b2")`
	`# Matches: ['1', '2']`
		re.fullmatch()	`re.fullmatch(r"\d+", "123")`
			`# Matches: 123`

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Matching with Captured Groups

```python
import re

text:str = "Il codice è: 123-ABC"
match = re.search(r"(\d+)-([A-Z]+)", text)

if match:
    print("Intera corrispondenza:", match.group(0)) # Output: "123-ABC"
    print("Gruppo 1 (numeri):", match.group(1)) # Output: "123"
    print("Gruppo 2 (lettere):", match.group(2)) # Output: "ABC"
```

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Substitution with Captured Groups

```python
import re

text:str = "123-ABC"
new_text:str = re.sub(r"(\d+)-([A-Z]+)", r"\2-\1", text)
print(new_text) # Output: "ABC-123"
```

💡 Here we use \1, \2 to **reverse the order** of the captured groups.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Practical RegEx Non-Capturing Example

```python
import re

text:str = "abcabcabc"
print("Cattura:", re.findall(r"(abc)+", text)) # Output ['abc']
print("Non cattura:", re.findall(r"(?:abc)+", text)) # Output ['abcabcabc’]
```

🧠 **Takeaway for Students**

- Use **capturing groups** only when you need to extract parts of the match.
- Use **non-capturing groups** when you just need grouping for repetition or logic — not for returning data.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

Practical RegEx Non-Capturing Example

1. `r"(abc)+"` — **Capturing Group**

- This pattern uses a **capturing group**: (abc) means we're grouping the string "abc".
- The + applies to the **entire group**, so it will match **one or more repetitions** of "abc" — for example, "abcabcabc".
- However, when using re.findall() in Python, **only the last match of the capturing group** is returned.
- Even though the full match is "abcabcabc", findall() **only gives back the content of the capturing group**, and since the group matched multiple times, only the last one ("abc") is returned.

Result: ['abc']

2. `r"(?:abc)+"` — **Non-Capturing Group**

- Here, (?:abc) creates a **non-capturing group** — it groups "abc" for repetition, but **does not save** any individual part of the match.
- This tells Python: *"Match the pattern, but don't remember specific groups for extraction."*
- Now re.findall() returns the **entire match**, because there's no capturing group to isolate.
- So "abcabcabc" is matched fully and **returned as is**.

Result: ['abcabcabc']

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY